

DATABASE SYSTE M S B OOTC A M P

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Part 1 —Practice Schema

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

customers:

id	name	email	signup_date
1	aman	aman@123gmail.com	2026-06-01
2	Ravi	ravi@231gmail.com	2026-06-02
3	Sneha	sneha254@gmail.com	2026-06-03
4	Karan	karan259@gmail.com	2026-06-04
5	Priya	priya449@gmail.com	2026-06-05
6	Neha	neha454@gmail.com	2026-06-06
7	Vikas	vikas154@gmail.com	2026-06-07
8	Anjali	anjali1454@gmail.com	2026-06-08
9	Rahul	rahul4878@gmail.com	2026-06-09
10	Meera	meera4587@gmail.com	2026-06-10
11	Suresh	suresh4826@gmail.com	2026-06-11
12	Pooja	pooja875@gmail.com	2026-06-12
13	Arjun	arjun_malhotra@gmail.com	2026-06-13
14	Divya	divya_kapoor@gmail.com	2026-06-14
15	Manish	manish_bhatia@gmail.com	2026-06-15

restaurants:

id	name	cuisine
1	kathiyavadi zyaka	Gujarti
2	Milon Hotel	Gujrati
3	Panjabi Tadka	Panjabi
4	Rajvadi Hotel	Gujrati
5	Hotel Dwarkesh	Gujarti
6	Brothers Cafe	Cafe
7	Umiyaji	Gujrati
8	Punjab Tandoor	Punjabi

orders:

id	customer_id	restaurant_id	total	status	created_at
1	1	1	25.50	Completed	2024-02-01
2	2	3	18.00	Completed	2024-02-15
3	3	2	32.75	Pending	2024-03-10
4	4	5	45.00	Completed	2024-03-22
5	5	4	22.00	Cancelled	2024-04-05
6	6	6	15.00	Completed	2024-04-20
7	8	8	28.50	Completed	2024-05-16
8	10	2	35.00	Completed	2024-06-12
9	12	3	42.00	Completed	2024-06-20
10	15	5	55.00	Completed	2024-06-25
11	1	6	18.50	Completed	2024-07-01
12	2	7	65.00	Completed	2024-07-08
13	3	8	27.00	Completed	2024-07-15
14	4	1	33.00	Completed	2024-07-22
15	5	2	29.00	Completed	2024-07-29
16	6	3	41.00	Completed	2024-08-05
17	8	5	52.00	Completed	2024-08-12
18	10	7	68.00	Completed	2024-08-19
19	12	1	24.00	Completed	2024-08-26
20	15	2	36.00	Completed	2024-09-02
21	1	3	40.00	Completed	2024-09-09
22	2	4	47.00	Completed	2024-09-16
23	3	5	25.00	Completed	2024-09-23
24	4	6	54.00	Completed	2024-09-30
25	5	7	19.00	Completed	2024-10-07
26	6	8	66.00	Completed	2024-10-14
27	8	1	28.00	Completed	2024-10-21
28	10	2	37.00	Completed	2024-10-28
29	12	3	49.00	Completed	2024-11-04
30	15	4	26.00	Completed	2024-11-11
31	1	5	55.00	Completed	2024-11-18
32	2	6	20.00	Completed	2024-11-25
33	3	7	67.00	Completed	2024-12-02
34	4	8	22.00	Completed	2024-12-09
35	5	1	30.00	Completed	2024-12-16
36	6	2	38.00	Completed	2024-12-23
37	8	3	44.00	Completed	2024-12-30
38	10	4	29.00	Completed	2025-01-06
39	12	5	53.00	Completed	2025-01-13
40	15	6	21.00	Completed	2025-01-20
41	1	7	68.00	Completed	2025-01-27
42	2	8	23.00	Completed	2025-02-03

accounts:

id	name	balance
1	Alice	2000.00
2	Bob	1000.00

Part 2 — Tasks

TASK 1 Transactions (ACID)

Using your accounts table:

- a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

Ans:

```
1 start transaction;
2 update accounts set balance = balance - 300 where id = 1;
3 update accounts set balance = balance + 300 where id = 2;
4 rollback ;
```

id	name	balance
1	Alice	2000.00
2	Bob	1000.00

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
start transaction;
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ 1 row affected. (Query took 0.0068 seconds.)

```
update accounts set balance = balance - 300 where id = 1;
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ 1 row affected. (Query took 0.0003 seconds.)

```
update accounts set balance = balance + 300 where id = 2;
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)

```
rollback;
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

- b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.
Ans:

```
1 start transaction;
2 update accounts set balance = balance - 300 where id = 1;
3 update accounts set balance = balance + 300 where id = 2;
4 commit ;
```

id	name	balance
1	Alice	1700.00
2	Bob	1300.00

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
start transaction;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row affected. (Query took 0.0067 seconds.)

```
update accounts set balance = balance - 300 where id = 1;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row affected. (Query took 0.0003 seconds.)

```
update accounts set balance = balance + 300 where id = 2;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)

```
commit;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

- c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Ans: The **Atomicity** property of ACID ensures that money is never "lost" during a transfer. Either both updates (debit and credit) succeed together, or if something fails, the entire transaction is rolled back, leaving balances unchanged. This guarantees that partial transfers cannot occur, preserving consistency of funds.

TASK 2 Joins

Using your customers and orders tables:

- a) Write an INNER JOIN that lists every customer who has placed at least one order.

Ans:

✓ Showing rows 0 - 9 (10 total, Query took 0.0005 seconds.)

```
SELECT distinct c.id, c.name, c.email FROM customers c INNER JOIN orders o ON c.id = o.customer_id;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

id	name	email
1	aman	aman@123gmail.com
2	Ravi	ravi@231gmail.com
3	Sneha	sneha254@gmail.com
4	Karan	karan259@gmail.com
5	Priya	priya449@gmail.com
6	Neha	neha454@gmail.com
8	Anjali	anjali1454@gmail.com
10	Meera	meera4587@gmail.com
12	Pooja	pooja875@gmail.com
15	Manish	manish_bhatia@gmail.com

- b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

Ans:

✓ Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

```
SELECT c.id, c.name, o.id FROM customers c LEFT JOIN orders o ON c.id = o.customer_id WHERE o.id IS NULL;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

id	name	id
7	Vikas	NULL
9	Rahul	NULL
11	Suresh	NULL
13	Arjun	NULL
14	Divya	NULL

- c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

Ans:

A marketing team would prefer query (b) because it identifies customers who have **never ordered**, allowing them to target these individuals with promotions or campaigns to encourage their first purchase.

TASK 3 Indexing

Using your orders table:

- a) Pick a column that is not currently indexed (for example status or created_at).

Ans: status

- b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

Ans:

Pending:

```
EXPLAIN ANALYZE SELECT *FROM orders
WHERE status = 'Pending';
```

Query #28 Execution time: 0.38ms

EXPLAIN

-> Filter: (orders.`status` = 'Pending') (cost=4.45 rows=42) (actual time=0.0192..0.0274 rows=1 loops=1) -> Table scan on orders (cost=4.45 rows=42) (actual time=0.0163..0.0224 rows=42 loops=1)

- c) Create an index on that column.

Ans:

```
CREATE INDEX idx_orders_status ON orders(status);
```

Query #29 Execution time: 18.12ms

There are no results to be displayed.

- d) Re-run the exact same query and record the new numbers.

Ans:

Pending:

```
EXPLAIN ANALYZE SELECT *FROM orders
WHERE status = 'Pending';
```

Query #30 Execution time: 0.37ms

EXPLAIN

-> Index lookup on orders using idx_orders_status (status = 'Pending') (cost=0.35 rows=1) (actual time=0.0124..0.0134 rows=1 loops=1)

- e) Submit a one-line comparison of before vs. after.

Ans:

Before: full table scan, 42 rows examined in ~0.38 ms (actual time=0.0163..0.0224 rows=42 loops=1).

After: index lookup, 1 row examined in ~0.37 ms (actual time=0.0124..0.0134 rows=1 loops=1).

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (
  SELECT id FROM customers WHERE signup_date >= 'some date');
```

✓ Showing rows 0 - 15 (16 total, Query took 0.0006 seconds.)

```
SELECT * FROM orders WHERE customer_id IN ( SELECT id FROM customers WHERE signup_date >= '2026-06-07' );
```

id	customer_id	restaurant_id	total	status	created_at
7	8	8	28.50	Completed	2024-05-16
8	10	2	35.00	Completed	2024-06-12
9	12	3	42.00	Completed	2024-06-20
10	15	5	55.00	Completed	2024-06-25
17	8	5	52.00	Completed	2024-08-12
18	10	7	68.00	Completed	2024-08-19
19	12	1	24.00	Completed	2024-08-26
20	15	2	36.00	Completed	2024-09-02
27	8	1	28.00	Completed	2024-10-21
28	10	2	37.00	Completed	2024-10-28
29	12	3	49.00	Completed	2024-11-04
30	15	4	26.00	Completed	2024-11-11
37	8	3	44.00	Completed	2024-12-30
38	10	4	29.00	Completed	2025-01-06
39	12	5	53.00	Completed	2025-01-13
40	15	6	21.00	Completed	2025-01-20

a) Rewrite it as a JOIN instead of a subquery.

Ans:

✓ Showing rows 0 - 15 (16 total, Query took 0.0005 seconds.)

```
SELECT o.* FROM orders o JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2026-06-07';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 50 | Filter rows: | Sort by key: None

Extra options

id	customer_id	restaurant_id	total	status	created_at
7	8	8	28.50	Completed	2024-05-16
8	10	2	35.00	Completed	2024-06-12
9	12	3	42.00	Completed	2024-06-20
10	15	5	55.00	Completed	2024-06-25
17	8	5	52.00	Completed	2024-08-12
18	10	7	68.00	Completed	2024-08-19
19	12	1	24.00	Completed	2024-08-26
20	15	2	36.00	Completed	2024-09-02
27	8	1	28.00	Completed	2024-10-21
28	10	2	37.00	Completed	2024-10-28
29	12	3	49.00	Completed	2024-11-04
30	15	4	26.00	Completed	2024-11-11
37	8	3	44.00	Completed	2024-12-30
38	10	4	29.00	Completed	2025-01-06
39	12	5	53.00	Completed	2025-01-13
40	15	6	21.00	Completed	2025-01-20

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Ans:

✓ Showing rows 0 - 15 (16 total, Query took 0.0005 seconds.)

```
SELECT o.id, o.total, c.name, c.signup_date FROM orders o JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2026-06-07';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: | Sort by key: None

Extra options

id	total	name	signup_date
7	28.50	Anjali	2026-06-08
8	35.00	Meera	2026-06-10
9	42.00	Pooja	2026-06-12
10	55.00	Manish	2026-06-15
17	52.00	Anjali	2026-06-08
18	68.00	Meera	2026-06-10
19	24.00	Pooja	2026-06-12
20	36.00	Manish	2026-06-15
27	28.00	Anjali	2026-06-08
28	37.00	Meera	2026-06-10
29	49.00	Pooja	2026-06-12
30	26.00	Manish	2026-06-15
37	44.00	Anjali	2026-06-08
38	29.00	Meera	2026-06-10
39	53.00	Pooja	2026-06-12
40	21.00	Manish	2026-06-15

- c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Ans:

```
-- original query---
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id IN (
  SELECT id FROM customers WHERE signup_date >= '2026-06-07');

-- join query without select * --
EXPLAIN ANALYZE
SELECT
  o.id,
  o.total,
  c.name,
  c.signup_date
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE c.signup_date >= '2026-06-07';
```

Query #34 Execution time: 0.41ms

EXPLAIN

-> Nested loop inner join (cost=19.1 rows=14) (actual time=0.0345..0.0941 rows=16 loops=1) -> Filter: (orders.customer_id is not null) (cost=4.45 rows=42) (actual time=0.0182..0.0295 rows=42 loops=1) -> Table scan on orders (cost=4.45 rows=42) (actual time=0.0176..0.0256 rows=42 loops=1) -> Filter: (customers.signup_date >= DATE'2026-06-07') (cost=0.251 rows=0.333) (actual time=0.00136..0.00139 rows=0.381 loops=42) -> Single-row index lookup on customers using PRIMARY (id = orders.customer_id) (cost=0.251 rows=1) (actual time=0.00115..0.00118 rows=1 loops=42)

Query #35 Execution time: 0.31ms

EXPLAIN

-> Nested loop inner join (cost=19.1 rows=14) (actual time=0.0294..0.088 rows=16 loops=1) -> Filter: (o.customer_id is not null) (cost=4.45 rows=42) (actual time=0.0141..0.0229 rows=42 loops=1) -> Table scan on o (cost=4.45 rows=42) (actual time=0.0137..0.0196 rows=42 loops=1) -> Filter: (c.signup_date >= DATE'2026-06-07') (cost=0.251 rows=0.333) (actual time=0.00138..0.00141 rows=0.381 loops=42) -> Single-row index lookup on c using PRIMARY (id = o.customer_id) (cost=0.251 rows=1) (actual time=0.00117..0.0012 rows=1 loops=42)

- Original Query (Subquery) → Execu on Time = 0.41 ms
- Rewritten Query (JOIN) → Execu on Time = 0.31 ms

The JOIN query executed slightly faster than the subquery because it directly combined the required tables. It also retrieved only the required columns, making the query more efficient.

Final Challenge — A Real-World Problem

Concepts used: this task combines everything from Tasks 2, 3, and 4 — Joins, Indexing, and Query Optimization — in one realistic scenario.

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

- Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

Ans:

Showing rows 0 - 3 (4 total. Query took 0.0003 seconds.) [created_at: 2025-01-13... - 2024-06-20...]

```
SELECT r.name AS restaurant_name, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 12 ORDER BY o.created_at DESC LIMIT 5;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Extra options

restaurant_name	total	status	created_at
Hotel Dwarkesh	53.00	Completed	2025-01-13
Punjabi Tadka	49.00	Completed	2024-11-04
kathiyavadi zyaka	24.00	Completed	2024-08-26
Punjabi Tadka	42.00	Completed	2024-06-20

- Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

Ans:

```
EXPLAIN ANALYZE
SELECT r.name AS restaurant_name,
       o.total,
       o.status,
       o.created_at
FROM orders o
JOIN restaurants r ON o.restaurant_id = r.id
WHERE o.customer_id = 12
ORDER BY o.created_at DESC
LIMIT 5;
```

Query #37 Execution time: 0.39ms

EXPLAIN

-> Limit: 5 row(s) (cost=9.7 rows=5) (actual time=0.0354..0.0405 rows=4 loops=1) -> Nested loop inner join (cost=9.7 rows=42) (actual time=0.035..0.0398 rows=4 loops=1) -> Sort: o.created_at DESC (cost=4.45 rows=42) (actual time=0.0303..0.0307 rows=4 loops=1) -> Filter: ((o.customer_id = 12) and (o.restaurant_id is not null)) (cost=4.45 rows=42) (actual time=0.0183..0.0257 rows=4 loops=1) -> Table scan on o (cost=4.45 rows=42) (actual time=0.0158..0.022 rows=42 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.274 rows=1) (actual time=0.00188..0.00191 rows=1 loops=4)

- c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

Ans:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0087 seconds.)

```
CREATE INDEX idx_orders_customer_created ON orders(customer_id, created_at);
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

- d) Re-run the same query and compare the new numbers to step (b).

Ans:

```
EXPLAIN ANALYZE
SELECT r.name AS restaurant_name,
       o.total,
       o.status,
       o.created_at
FROM orders o
JOIN restaurants r ON o.restaurant_id = r.id
WHERE o.customer_id = 12
ORDER BY o.created_at DESC
LIMIT 5;
```

Query #39 Execution time: 0.59ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.55 rows=4) (actual time=0.0357..0.0495 rows=4 loops=1) -> Nested loop inner join (cost=2.55 rows=4) (actual time=0.0349..0.0483 rows=4 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.15 rows=4) (actual time=0.0268..0.0352 rows=4 loops=1) -> Index lookup on o using idx_orders_customer_created (customer_id = 12) (reverse) (cost=1.15 rows=4) (actual time=0.0258..0.0338 rows=4 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.275 rows=1) (actual time=0.00271..0.00275 rows=1 loops=4)

The execution time after creating the index was slightly lower than in step (b). The index helped the database find the required records more efficiently, although the improvement was small because the dataset is small.

- e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Ans:

✓ Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.) [created_at: 2025-01-13... - 2024-06-20...]

```
SELECT r.name AS restaurant_name, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 12 ORDER BY o.created_at DESC LIMIT 5;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Extra options

restaurant_name	total	status	created_at	▼ 1
Hotel Dwarkesh	53.00	Completed	2025-01-13	
Panjabi Tadka	49.00	Completed	2024-11-04	
kathiyavadi zyaka	24.00	Completed	2024-08-26	
Panjabi Tadka	42.00	Completed	2024-06-20	

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Ans:

Before creating the index, the query took slightly more time to execute. After creating the index, the execution time improved slightly because the database located the required records more efficiently. Since the dataset is small, the improvement is minimal, but with a larger amount of data, the index would significantly improve query performance.

The index was created on `customer_id` and `created_at` because these columns are used in the `WHERE` and `ORDER BY` clauses. The index reduces the amount of data the database needs to search and sort, making the query more efficient. With a larger amount of data, the index provides much greater performance improvement because it avoids scanning the entire table.